

CS360 Introduction to Database - Final Problem Sheet

- **Date & Time:** June 11 (Wednesday), 13:00 ~ 15:30
 - **Separate answer sheet:** This is a question sheet, and all grading will be based on the answers on the separate answer sheet only. Write clear answers on the answer sheet please.
 - **Name on answer sheet:** You must write your name on the answer sheet.
 - **No Electronic Devices:** The use of any electronic devices is strictly prohibited.
 - **Questions During the Exam:** If you have any questions, please raise your hand.
 - **Deduction Policy:** The score for each Problem is non-negative. (i.e., $\max(\text{sum}(\text{score}), 0)$)
-

Problem 1 (5 points)

Determine whether the following statements are True (T) or False (F).

(1 point for each correct, -1 point for each incorrect)

1. In a B-tree, inserting a new key can increase the depth of the tree by causing a split that propagates up to the root.
2. RAID 5 offers a good balance between cost, storage efficiency, and read performance, but its write performance is slower due to parity overhead.
3. Given a relation R with $B(R) = 20$ blocks and a memory size $M = 40$, it is possible to perform duplicate elimination on R using a one-pass algorithm.
4. In a query with a condition like $\text{upper}(r.A) = \text{upper}(s.A)$, it is possible to use a hash join or merge join if the deterministic $\text{upper}()$ function is first applied to both $r.A$ and $s.A$ before performing join.
5. Nested-loops join algorithm is optimal for the task on a database stored in a column-oriented manner.

Problem 2 (8 points)

Data such as images and documents can be represented as high-dimensional vectors, and a vector database is a database system designed to store these vectors and enable efficient retrieval. One of the core operations in a vector database is performing nearest neighbor search, which finds the vectors that are closest to a given query vector based on a specified distance metric. Answer the following questions. **No explanation is needed.**

(1) Suppose each block has a size of 4096 bytes and requires a 12-byte block header. Assume each vector is a 128-dimensional vector of 4-byte floats. How many vectors can be stored in a single page? No vector spans across multiple blocks. **(2 points)**

(2) For (1), suppose that 10,000 vector data entries are tightly packed into consecutive blocks with maximum space efficiency. (Still, no vector is allowed to span across multiple blocks.) The simplest way to perform the previously mentioned nearest neighbor search is to scan all 10,000 vectors. In this case, how many blocks need to be read? In other words, what is the total number of blocks required to store the entire vector dataset? **(2 points)**

(3) Since the brute-force method in (2) becomes computationally expensive as the dataset grows, real-world applications often perform nearest neighbor search approximately. One such method organizes the vectors as vertices in a graph, where nearby vertices are connected via edges according to certain rules. In this case, the data stored for each vector entry includes both the vector itself and its corresponding graph topology, as follows (assume that each vertex has 32 out-edges and each neighbor is represented as a 4-byte integer):

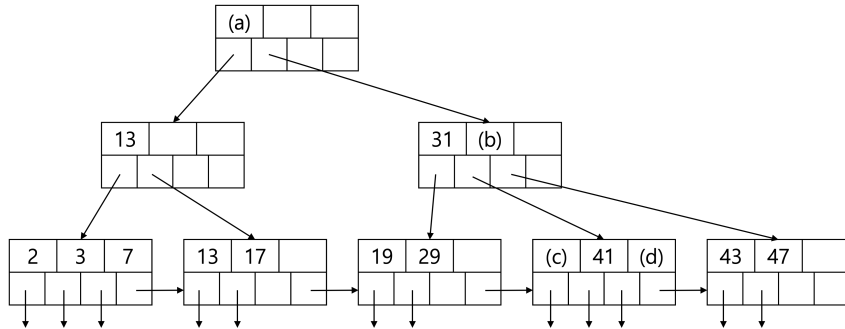
- neighbor list (`int[32]`)
- vector data (`float[128]`)

How many vector entries can be stored in a single page? Assume that the block size is 4096 bytes and each block has a 12-byte header, as in (1). **(2 points)**

(4) Suppose that using the graph-based approach in (3), an approximate search result can be obtained by traversing 50 hops (i.e., accessing 50 vertex data entries) for the same-sized dataset from (2). In this case, what are the **minimum** and **maximum** number of blocks that may need to be read for a single search? Assume that during the search process, if a previously accessed vertex and the current vertex reside in the same block, there is no need to read that block again. **(2 points)**

Problem 3 (12 points)

Consider the B⁺-tree shown below. The B⁺-tree in this problem follows the semantics covered in class and homework. Note that duplicate keys are not allowed, and all keys are integers. **No explanation is needed.**



(1) What are the values for (a)-(d)? (2 points)

(2) Suppose one of the 12 keys currently stored in the leaf nodes of the above B⁺-tree is deleted. For each key, indicate whether its deletion results in changes to nodes other than the leaf node containing the key, including any structural changes such as merge, by marking **Yes** or **No**. Note that each key deletion should be considered independently, with the initial tree reset to the one shown in the above figure for every deletion. (6 points, -0.5 points for each wrong answer)

(3) Assume that, in the current system, three nodes of the above B⁺-tree are cached in main memory. The **hit rate** is defined as below:

$$\text{hit rate} = \frac{\# \text{ of accesses to cached nodes}}{\# \text{ of total node accesses}}$$

In this problem, we use a **user-defined static caching** policy, where the user selects 3 nodes in advance, and these selected nodes remain cached throughout the execution of the workload. Given the workload below, compute the **maximum** and **minimum** possible cache hit rates achievable under user-defined static caching. (4 points)

- Workload: {lookup key 13, lookup key 17, lookup key 19, lookup key 41, lookup key 43}

Problem 4 (10 points)

Answer the below questions. **No explanation is needed.**

(1) Fill in the blanks (a)–(e) in the passage below by choosing from the following words. Each word may be used more than once. **(2 points, -0.5 points for each wrong answer)**

sparse, dense, first, last

The first level of secondary index is always (a) . Second and higher levels must be (b) . According to the lecture slides, linear hash table uses the (c) bits of the hash value, while extensible hash table uses the (d) bits.

(2) Encode the following bit string using the run-length encoding method covered in class. **(2 points)**

00000001110001

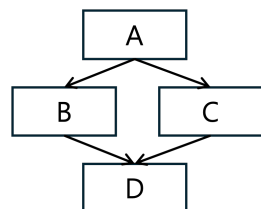
(3) Decode the following bit string using the run-length encoding method covered in class. The decoded bit string should contain **exactly 10 bits**. **(2 points)**

110110

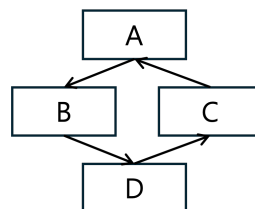
(4) In the pointer swizzling scheme, when a block is moved from memory back to disk, to ensure that the block can be safely written back, the following two conditions must both be satisfied:

1. Any pointers within the block must be unswizzled, meaning that the memory addresses of the pointers are replaced by the corresponding database addresses.
2. The block must be in an unpinned state, meaning that any pointers referencing that block have been unswizzled first.

In this problem, assume that the expression “block eviction” means checking whether the block is in an unpinned state and, if so, unswizzling all pointers within the block. For each scenario in the figure, give one possible block eviction order so that all pages are evicted. In the figure, each rectangle represents a block, all blocks are initially in memory, and all in-memory pointers have been swizzled; an arrow from block A to block B means A holds a pointer to a data item in B. If no valid eviction order exists, answer “impossible.” (Example answer: $A \rightarrow B \rightarrow C \rightarrow D$) **(4 points)**



(a)



(b)

Problem 5 (16 points)

In this problem, we have two relations, $R(X, Y)$ and $S(Y, Z)$. Here, $R.Y$ and $S.Y$ are all clustered. Relation R and S require B_R and B_S blocks respectively for storage. Let M denote the total number of blocks available in the system. We only consider the following join implementation methods: one-pass join (J), block nested-loop join (NLJ), and sort-merge join (SMJ). Assume that $B_R, B_S, M \in \mathbb{N}$.

Let $J_{op}(B_R, B_S, M)$ be the **subset** of the join methods $\{J, NLJ, SMJ\}$ that result in the **minimum** disk I/O cost when joining R and S given B_R, B_S, M . If multiple methods have the same minimal cost, return the set of such methods. We do not consider the cost of writing the query answer. Other assumptions and all join methods work as the same as we have studied in the class.

Find all values of M and the corresponding $J_{op}(100, 50, M)$ such that $J_{op}(100, 50, M) \neq J_{op}(100, 50, M+1)$ for $11 \leq M \leq 101$. List the rows in descending order of M . Note that $J_{op}(100, 50, M)$ returns a ***set*** of operator(s). **No explanation is needed. (16 points, 2 points each)**

M	$J_{op}(100, 50, M)$
50	{J}

Problem 6 (10 points)

In this problem, we have two relations, $R(X, Y)$ and $S(Y, Z)$. Assume that both $R.Y$ and $S.Y$ are clustered. Our goal is to compute the disk I/O cost of the two-pass selection operation and the index-based join operation. All methods behave exactly as covered in class. Furthermore, we do not consider the cost of writing the query answer, and we assume that the cost to access the index is zero. **No explanation is needed.**

(1) Consider the following situation:

- Both R and S contain 200,000 tuples.
- $R.Y$ has 10 unique values, and $S.Y$ has 200 unique values. i.e., $V(R, Y) = 10, V(S, Y) = 200$.
- $B(R) = 2000$ and $B(S) = 100$.
- Both $R.Y$ and $S.Y$ do **not** have indices.

Compute the disk I/O cost for (a) $\sigma_{Y=0}(R)$, and (b) $\sigma_{Y=0}(S)$ **(2 points)**

(2) Under the same conditions as in (1), assume that both $R.Y$ and $S.Y$ **have** indices and are clustered. Compute the disk I/O cost for (a) $\sigma_{Y=0}(R)$, and (b) $\sigma_{Y=0}(S)$ **(2 points)**

(3) Under the same conditions as in (1), assume that **only** $R.Y$ **has** index and is clustered (**i.e., $S.Y$ does not have index and not clustered**). We use the index on $R.Y$ to perform the index-based join $R(X, Y) \bowtie S(Y, Z)$. What is the disk I/O cost? **(3 points)**

(4) Under the same conditions as in (1), assume that **only** $S.Y$ **has** index and is clustered (**i.e., $R.Y$ does not have index and not clustered**). We use the index on $S.Y$ to perform the index-based join $R(X, Y) \bowtie S(Y, Z)$. What is the disk I/O cost? **(3 points)**

Problem 7 (8 points)

In this problem, we aim to estimate the result sizes of two SQL queries. For convenience, we refer to the relation for **HansaLuft A.G.** as **HL**, and for **Asian Airlines** as **AAL**. The actual data of the two relations, **HL** and **AAL**, are shown in Table 1. All cardinality estimation methods used in this problem operate exactly as covered in the class. **No explanation is needed.**

HL		AAL	
DepIATA	Dist	DepIATA	Dist
ICN	5000	ICN	7000
JFK	3000	FRA	5000
FRA	1000	HND	750
MUC	500	PVG	500
FRA	500	ICN	5000
FRA	6000	PUS	300
LHR	1000	JFK	7000
MUC	6000	PUS	500

Table 1: A departure airport (**DepIATA**) and travel distance (**Dist**) for two different fictional airlines, **HansaLuft A.G.** (left), and **Asian Airlines** (right).

(1) Consider the following SQL query:

```
SELECT HL.Dist, AAL.Dist
FROM HL, AAL
WHERE HL.DepIATA = AAL.DepIATA;
```

Based on the data in Table 1, calculate the actual number of tuples returned by this SQL query. **(1 point)**

(2) Now, estimate the number of tuples returned by the same query in (1), but using the most-frequent-values histogram method. For this estimation, assume the following: (a) All values that occur in two or more rows are explicitly listed, and any remaining values are grouped into a category labeled “others”. (b) There are 10 possible **DepIATA** values for both relations, and in the estimation, all values not listed as most frequent are assumed to have equal probability within the “others” category (i.e., uniformly spreaded). If the answer has more than two digits after the decimal point, round it to the nearest value at the **second decimal place**. **(3 points)**

(3) Consider the following SQL query:

```
SELECT HL.DepIATA
FROM HL
WHERE HL.Dist > 2500;
```

Based on the data in Table 1, calculate the **actual number** of tuples returned by this SQL query. **(1 point)**

(4) Estimate the result size of the query in (3) using an equal-width histogram. Assume the width of each bucket is 2000 and that the buckets start from 0. For this estimation, assume uniform distribution within each bucket of the histogram. If the answer has more than two digits after the decimal point, round it to the nearest value at the **second decimal place**. **(3 points)**

Problem 8 (9 points)

Consider the relational algebra on ***bags*** (not sets). This problem asks you to optimize the query by pushing selection. **Based on two tables with $R(A, B)$ and $S(B, C, D)$, push the selection as much as possible. No explanation is needed.**

(1) Push the selection and give an optimized expression that best minimizes the input tuples to natural join ($\bowtie$) in the expression: $\sigma_{A=a1 \wedge D>d1}(R \bowtie S)$. **(2 points)**

$$\sigma_{A=a1 \wedge D>d1}(R \bowtie S) = \underline{\hspace{15cm}}$$

(2) Push the selection and give an optimized expression that best minimizes the input tuples to natural join ($\bowtie$) in the expression: $\sigma_{(A=a1 \vee C=c1) \wedge D>d1}(R \bowtie S)$. **(3 points)**

$$\sigma_{(A=a1 \vee C=c1) \wedge D>d1}(R \bowtie S) = \underline{\hspace{15cm}}$$

(3) Push the selection and give an optimized expression that best minimizes the input tuples to the left outer join ($\Join$) on the expression: $\sigma_{B=b1 \wedge D>d1}(R \Join S)$. **(4 points)**

$$\sigma_{B=b1 \wedge D>d1}(R \Join S) = \underline{\hspace{15cm}}$$

Problem 9 (12 points)

This problem asks you to optimize a correlated subquery into an uncorrelated subquery by unnesting it. Answer the questions given. **No explanation is needed.**

Students		Enrollment		
<i>SID</i> (not null)	<i>name</i> (not null)	<i>SID</i> (not null)	<i>courseCode</i> (not null)	<i>grade</i> (not null)
19990001	James	19990001	CS204	B+
19990002	Hana	19990002	CS360	A+
19990003	Paul	19990003	EE210	A0
19990004	Sarah			

Figure 1: Example tables (grade is comparable and follows the order $A+ > A0 > B+$).

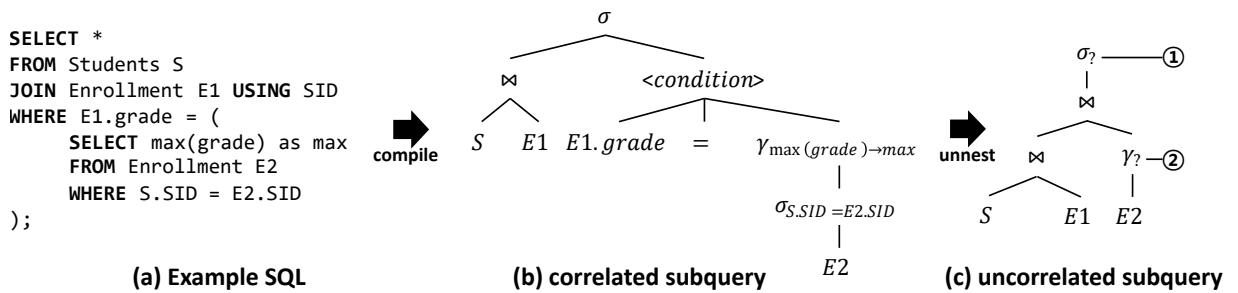


Figure 2: Unnesting correlated subquery with aggregation.

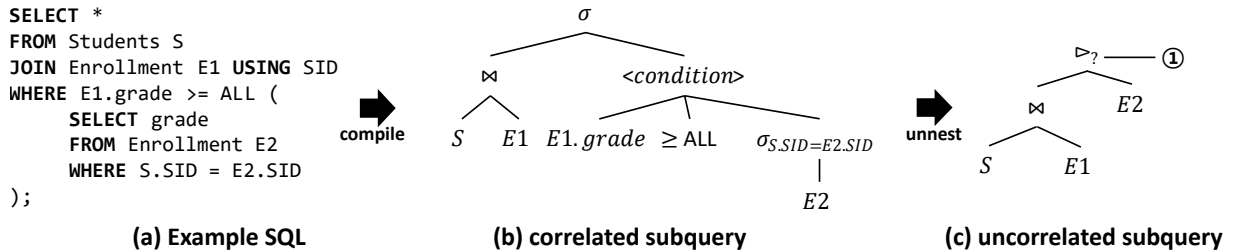


Figure 3: Unnesting correlated subquery with anti-join.

(1) Consider the correlated subquery plan in Figure 2(b) executed on the database in Figure 1. How many times should the expression $S.SID = E2.SID$ be evaluated? **(2 points)**

(2) Try to unnest the plan in Figure 2(b) into Figure 2(c). Fill in ① and ②. **(5 points)**

(3) Figure 3(a) represent SQL with the same semantics as SQL in Figure 2(a) but without aggregation, and using $\geq ALL$. However, this query still contains a correlated subquery, as shown in Figure 3(b). So, by using the anti-theta-join, try to unnest Figure 3(b) into Figure 3(c). The anti-theta-join of R and S , denoted as $R \triangleright_{\theta} S$, is the set of tuples in R that cannot be theta-join ($\bowtie_{\theta}$) with any tuple in S . Fill in ①. **(5 points)**

Problem 10 (10 points)

This problem asks you to perform end-to-end optimization from logical planning to physical planning for a JOIN query. Based on three tables with schema $R(A, B)$, $S(B, C)$, and $T(C, D)$, answer the given questions. **No explanation is needed.**

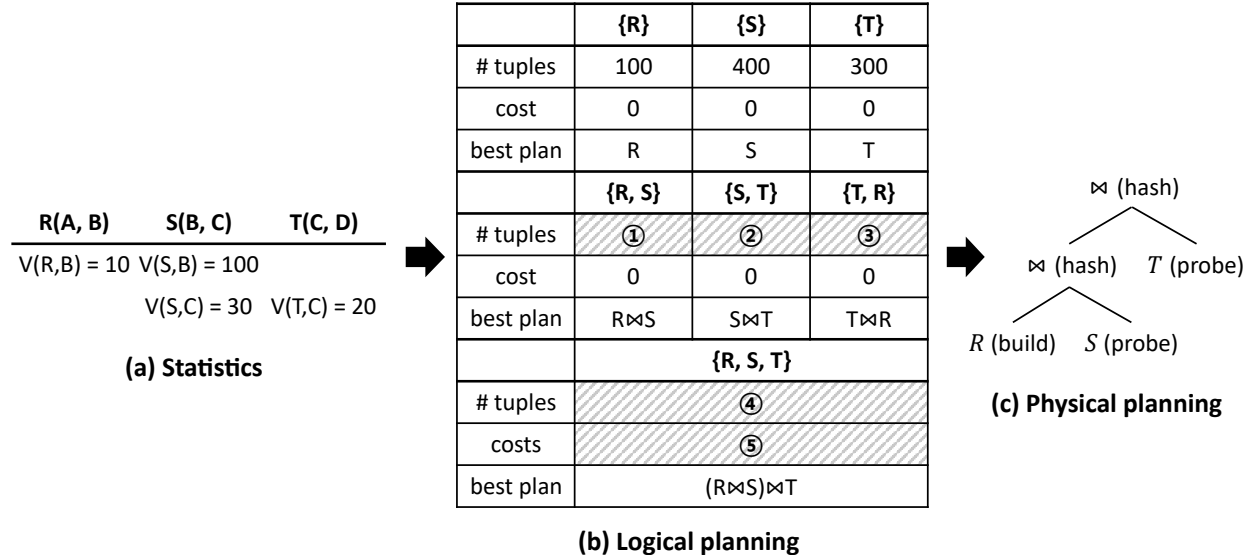


Figure 4: Example of logical and physical optimization flow.

(1) Figure 4(b) shows the logical planning step using dynamic programming method based on the statistics in Figure 4(a). Fill all empty cells (① - ⑤). **(5 points)**

(2) Figure 4(c) shows the physical planning step that determines the type of join and the execution method in the logical plan selected in Figure 4(b). For simplicity, we assume only hash joins where the left table is built and the right table is probed, and all hash buckets are always uniformly partitioned. Give the minimum disk I/O by finding the optimal physical plan by considering whether to pipeline or materialize and whether it is 1-pass or 2-pass. Let $M = 21$ and the size of the block is 1 (that is, $B(R) = \# \text{ tuples in } R$). **(5 points)**